

CSRF Protection Hardening — H.A.M.M.E.R. POS

Date: 2026-05-12
Author: Security Engineering
Scope: All mutating API routes (POST / PUT / PATCH / DELETE)

Summary

- The CSRF protection system was incomplete:
- **middleware.ts** only checked that the `x-csrf-token` header was present — it never validated the token value.
 - The `requireCsrf()` function in `src/modules/security/csrf.ts` properly validates tokens (hashes + DB lookup + expiry check), but **43 mutating routes** did not call it.
 - Only 13 routes (auth, cashier, sales, transport) had real CSRF validation.

This changeset ensures **every mutating route** now calls `requireCsrf()` for full token validation, while preserving all existing functionality.

Architecture (Defense-in-Depth)

Request → middleware.ts (shallow: header exists?) → route handler (deep: requireCsrf validates hash + DB + expiry)

Layer	What it checks	Fails with
<code>middleware.ts</code>	Header <code>x-csrf-token</code> is present	403 <code>MISSING_CSRF_TOKEN</code>
<code>requireCsrf()</code>	Token is valid, hashed match in DB, not expired	403 <code>INVALID_CSRF_TOKEN</code>

Files Modified

New Files

File	Purpose
<code>src/modules/security/route-protection.ts</code>	<code>requireAuthAndCsrft()</code> helper combining auth + CSRF
<code>scripts/verify-csrf-protection.ts</code>	Automated verification script
<code>CSRF_PROTECTION_CHANGES.md</code>	This document

Modified Files (middleware)

File	Change
<code>middleware.ts</code>	Added documentation comments explaining shallow vs deep CSRF validation

Modified Route Files (43 routes hardened)

Route	Methods	File
/api/ai-insights/refresh	POST	src/app/api/ai-insights/refresh/route.ts
/api/analytics/classify	POST	src/app/api/analytics/classify/route.ts
/api/approvals/[id]	PATCH	src/app/api/approvals/[id]/route.ts
/api/branch-config	PUT	src/app/api/branch-config/route.ts
/api/cash-closure	POST	src/app/api/cash-closure/route.ts
/api/cash-closure/reopen	POST	src/app/api/cash-closure/reopen/route.ts
/api/catalog/categories	POST	src/app/api/catalog/categories/route.ts
/api/catalog/categories/[id]	PATCH	src/app/api/catalog/categories/[id]/route.ts
/api/catalog/products	POST	src/app/api/catalog/products/route.ts
/api/catalog/products/[id]	PATCH	src/app/api/catalog/products/[id]/route.ts
/api/employees	POST	src/app/api/employees/route.ts
/api/employees/[id]	PUT, DELETE	src/app/api/employees/[id]/route.ts
/api/expenses	POST	src/app/api/expenses/route.ts
/api/expenses/[id]	PUT, DELETE	src/app/api/expenses/[id]/route.ts
/api/inventory/adjustments	POST	src/app/api/inventory/adjustments/route.ts
/api/inventory/movements	POST	src/app/api/inventory/movements/route.ts

Route	Methods	File
/api/master/analytics/abc-xyz/[id]	PUT	src/app/api/master/analytics/abc-xyz/[id]/route.ts
/api/master/cash-boxes/[id]/toggle	PATCH	src/app/api/master/cash-boxes/[id]/toggle/route.ts
/api/master/catalog/products/[id]/cleanup	POST	src/app/api/master/catalog/products/[id]/cleanup/route.ts
/api/master/discounts	POST	src/app/api/master/discounts/route.ts
/api/master/discounts/[id]	PUT, DELETE	src/app/api/master/discounts/[id]/route.ts
/api/master/inventory/import	POST	src/app/api/master/inventory/import/route.ts
/api/master/purchase-orders	POST	src/app/api/master/purchase-orders/route.ts
/api/master/purchase-orders/[id]/approve	POST	src/app/api/master/purchase-orders/[id]/approve/route.ts
/api/master/purchase-orders/[id]/cancel	POST	src/app/api/master/purchase-orders/[id]/cancel/route.ts
/api/master/transfers	POST	src/app/api/master/transfers/route.ts
/api/master/transfers/[id]/approve	POST	src/app/api/master/transfers/[id]/approve/route.ts
/api/master/transfers/[id]/cancel	POST	src/app/api/master/transfers/[id]/cancel/route.ts
/api/master/users	POST	src/app/api/master/users/route.ts
/api/master/users/[id]	PATCH	src/app/api/master/users/[id]/route.ts
/api/master/users/[id]/memberships	POST	src/app/api/master/users/[id]/memberships/route.ts

Route	Methods	File
/api/master/users/[id]/memberships/[membershipId]	PATCH, DELETE	src/app/api/master/users/[id]/memberships/[membershipId]/route.ts
/api/payroll/calculate	POST	src/app/api/payroll/calculate/route.ts
/api/pricing/config	POST	src/app/api/pricing/config/route.ts
/api/system-admin/role-config	PUT	src/app/api/system-admin/role-config/route.ts
/api/system-admin/settings	PUT	src/app/api/system-admin/settings/route.ts
/api/timber	POST	src/app/api/timber/route.ts
/api/timber/[id]	PUT, DELETE	src/app/api/timber/[id]/route.ts
/api/timber/calculate	POST	src/app/api/timber/calculate/route.ts
/api/timber/pricing	PUT	src/app/api/timber/pricing/route.ts
/api/timber/trips	POST	src/app/api/timber/trips/route.ts
/api/timber/trips/[id]	PUT, PATCH	src/app/api/timber/trips/[id]/route.ts
/api/warehouse/dispatch/[orderId]/dispatch	POST	src/app/api/warehouse/dispatch/[orderId]/dispatch/route.ts

Previously Protected Routes (13 — unchanged)

Route	Methods
/api/auth/change-password	POST
/api/auth/logout	POST
/api/cashier/cash-sessions/close-request	POST
/api/cashier/cash-sessions/close	POST
/api/cashier/cash-sessions/open	POST
/api/cashier/payments	POST
/api/sales/orders	POST
/api/sales/orders/[id]/direct-sale	POST
/api/sales/orders/[id]/lines	POST
/api/sales/orders/[id]/lines/[lineId]	PATCH, DELETE
/api/sales/orders/[id]/submit	POST
/api/transport	POST
/api/transport/[id]	PATCH

Justified Exceptions

Route	Method	Reason
/api/auth/login	POST	No session exists yet — the user hasn't authenticated, so there's no session to bind a CSRF token to.
/api/auth/csrf	GET	Token generation endpoint — GET-only, no mutation.
/api/auth/session	GET	Session reading endpoint — GET-only, no mutation.

Before / After Examples

Before (vulnerable):

```
// src/app/api/employees/route.ts
export async function POST(request: Request) {
  try {
    const session = await getCurrentSession();
    assertAuthenticated(session);
    assertMaster(session);
    // ❌ No CSRF validation – any origin can forge this request
    const body = await request.json();
    // ...
  }
}
```

After (hardened):

```
// src/app/api/employees/route.ts
import { requireCsrf } from "@modules/security/csrf";

export async function POST(request: Request) {
  try {
    const session = await getCurrentSession();
    assertAuthenticated(session);
    await requireCsrf(request, session); // ✅ Validates token hash + DB + expiry
    assertMaster(session);
    const body = await request.json();
    // ...
  }
}
```

Helper available for new routes:

```
import { requireAuthAndCsrf } from "@modules/security/route-protection";

export async function POST(request: Request) {
  try {
    const session = await requireAuthAndCsrf(request); // Auth + CSRF in one call
    // ...
  }
}
```

Verification

Run the automated verification script:

```
npx tsx scripts/verify-csrf-protection.ts
```

Expected output:

```
Total mutating routes scanned: 57
✅ Protected (requireCsrf):      56
🔒 Justified exceptions:         1
❌ UNPROTECTED:                  0

✅ ALL MUTATING ROUTES ARE CSRF-PROTECTED (or justified exceptions).
```


Manual testing:

```
# 1. Start the dev server
npm run dev

# 2. Login and get a CSRF token
curl -c cookies.txt http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"secret"}'

curl -b cookies.txt http://localhost:3000/api/auth/csrf
# → {"csrfToken":"abc123..." }

# 3. Test a protected route WITHOUT CSRF (should fail)
curl -b cookies.txt http://localhost:3000/api/employees \
  -H "Content-Type: application/json" \
  -d '{"name":"Test"}'
# → 403 MISSING_CSRF_TOKEN (blocked by middleware)

# 4. Test with invalid CSRF (should fail)
curl -b cookies.txt http://localhost:3000/api/employees \
  -H "Content-Type: application/json" \
  -H "x-csrf-token: invalid" \
  -d '{"name":"Test"}'
# → 403 INVALID_CSRF_TOKEN (blocked by requireCsrf in route)

# 5. Test with valid CSRF (should succeed)
curl -b cookies.txt http://localhost:3000/api/employees \
  -H "Content-Type: application/json" \
  -H "x-csrf-token: abc123..." \
  -d '{"name":"Test"}'
# → 200/201 (request proceeds)
```

Frontend Compatibility

The project already has `apiFetch()` in `src/lib/http.ts` which:

1. Automatically fetches a CSRF token from `/api/auth/csrf` before mutating requests
2. Attaches it as `x-csrf-token` header
3. On 403 `INVALID_CSRF_TOKEN`, refreshes the token and retries once

Frontend components that use `apiFetch()` will work without changes.

Components that use raw `fetch()` directly should be migrated to `apiFetch()` to benefit from automatic CSRF token management.

Next Steps

1. **Frontend audit:** Search for raw `fetch()` calls to mutating endpoints and migrate to `apiFetch()`
2. **CI integration:** Add `npm run scripts/verify-csrf-protection.ts` to the CI pipeline
3. **Token rotation:** Consider rotating CSRF tokens after each use (single-use tokens)
4. **SameSite cookies:** Verify session cookies have `SameSite=Lax` or `Strict` for additional protection